THE BOMBAY SALESIAN SOCIETY'S

DON BOSCO INSTITUTE OF TECHNOLOGY, MUMBAI

(An Autonomous Institute Affiliated to the University of Mumbai)

Department of Information Technology

CONTINUOUS ASSESSMENT 3 (CA-3)

Software Engineering — TE IT

Web Application Testing

Individual Report

APPLICATION TESTED

ERPNext v16.51.1

Open-source ERP on the Frappe Framework, hosted locally at http://localhost:8080

https://github.com/frappe/erpnext

Name	Deviprasad Shetty
Roll Number	55
Class and Division	TE IT — Division A
Subject	Software Engineering
Marks	20
Date of submission	29 October 2026

Declaration of Individual Work

I declare that this report is the result of my own individual work for CA-3 of Software Engineering (TE IT). I selected the open-source application named below, hosted it on my own computer, designed the test cases myself, executed them against my own local instance, and recorded the actual results shown in this report.

Every screenshot in this report was captured by me from the instance running on my machine at http://localhost:8080. No screenshot has been taken from the internet, from another student, or from a public or college server. No group work was involved at any stage.

All security testing was performed only against my own local copy of the application, as instructed in the assignment brief. I did not scan, probe or otherwise test any public website, third-party system or institute server.

The application tested is ERPNext v16.51.1, an open-source ERP distributed under the GNU General Public License v3, obtained from https://github.com/frappe/erpnext. The tools used are named with their versions in Section 6, and all of them are free to use.

Signature	**Date**
_____	29 October 2026
Name: Deviprasad Shetty	Roll Number: 55

1. Introduction and Application Chosen

1.1 Purpose of this report

This report documents the testing of a real, open-source web application running on my own machine. The brief asks for one application with a login, at least three modules containing forms, and a database; a test plan covering six areas; at least fifteen executed test cases; at least three logged defects; and screenshot evidence for every claim. This report follows that structure. The objective was not to confirm that the software works, but to find out where it does not — and to record what actually happened rather than what should have happened.

1.2 Application chosen and justification

The brief recommends OpenCart and lists Taiga, Akaunting and BookStack as alternatives, while allowing any open-source web application that can be hosted locally. I chose ERPNext v16.51.1 for four reasons.

- It satisfies every structural requirement: a login with role-based access control, dozens of modules built from data-entry forms, and a MariaDB database behind all of them. It is also far richer than a storefront for form testing — a single Item carries mandatory fields, conditional fields, numeric allowances, child tables and validation rules, which is exactly the surface this course wants examined.
- It exposes a documented REST API under /api/resource and /api/method, so the API test area can be tested properly rather than simulated. A shop front with no API would have made that area untestable.
- It ships an official Docker Compose file, so the whole stack comes up with one command and the version is pinned by image tag. That makes the environment reproducible and the version claim in this report verifiable from a screenshot.

The trade-off is honest to state: ERPNext has no storefront, no shopping cart and no checkout, so the sample cases in the brief that assume a shop do not apply literally. I replaced them with the equivalent forms in the Stock module — item creation, editing, list filtering and Stock Entry — which exercise the same class of logic: mandatory fields, duplicate keys, numeric boundaries and unit changes. All six required test areas are covered.

Version: ERPNext v16.51.1 on the Frappe Framework v16, running from the image frappe/erpnext:v16.51.1. The version is visible in the running containers in Figure S02. Source: https://github.com/frappe/erpnext. Documentation: https://docs.frappe.io/erpnext.

1.3 What was tested

Testing concentrated on the Stock module of the Desk interface — the Item list, the Item form and Stock Entry — together with authentication, the REST API, page load performance, responsive layout and cross-browser behaviour. Seventeen cases were designed and all seventeen were executed. Accounting, Manufacturing, Payroll and the selling pipeline were left out of scope because their depth would not add new defect classes within the time available.

2. Local Hosting Setup

The application was hosted on my own laptop using Docker rather than XAMPP, because ERPNext is a Python application on MariaDB with Redis and background workers — XAMPP, which targets PHP and MySQL, cannot host it. The brief permits either option, and the Docker route used here is Option B in the brief.

Steps followed, with the figure that proves each one:

1. Installed Docker Engine on the laptop and confirmed both client and server respond with docker version. The client reported version 29.9.1, API 1.56, Go 1.26.9, build dfc520d, linux/amd64.
2. Attempted docker ps and was refused: "permission denied while trying to connect to the docker API at unix:///var/run/docker.sock". The cause is that the logged-in user was not in the docker group, so the root-owned socket was unreadable. Figure S01 captures the working client together with this failure — evidence that Docker is installed and that the failure is a host permission problem, not a broken installation.
3. Fixed the permission by adding the user to the docker group with sudo usermod -aG docker $USER, refreshing the group with newgrp docker, and confirming with docker ps.
4. Obtained the official compose file and started the stack with docker compose -f pwd.yml up -d. Eleven services came up: the configurator and create-site containers ran once and exited successfully (the site was created), while backend, frontend, websocket, queue-long, queue-short, scheduler, db, redis-cache and redis-queue stayed running. The frontend publishes port 8080. Figure S02 shows the full sequence.
5. Opened http://localhost:8080, signed in as the Administrator account, and reached the Desk home page. Figure S03.

Figures S01 and S02 record steps 2 and 4 respectively: the first shows the Docker client working while the daemon socket refuses the connection, and the second shows the whole stack coming up and the site being created.

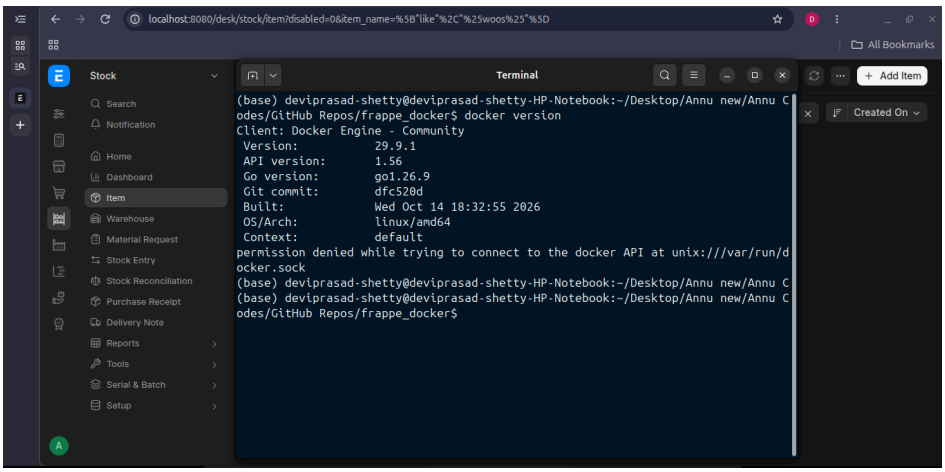


Figure S01 — *Step 2 — Docker Engine 29.9.1 is installed and the client responds, but docker ps is refused: "permission denied while trying to connect to the docker API at unix:///var/run/docker.sock". The installation is sound; the user is simply not in the docker group yet.*

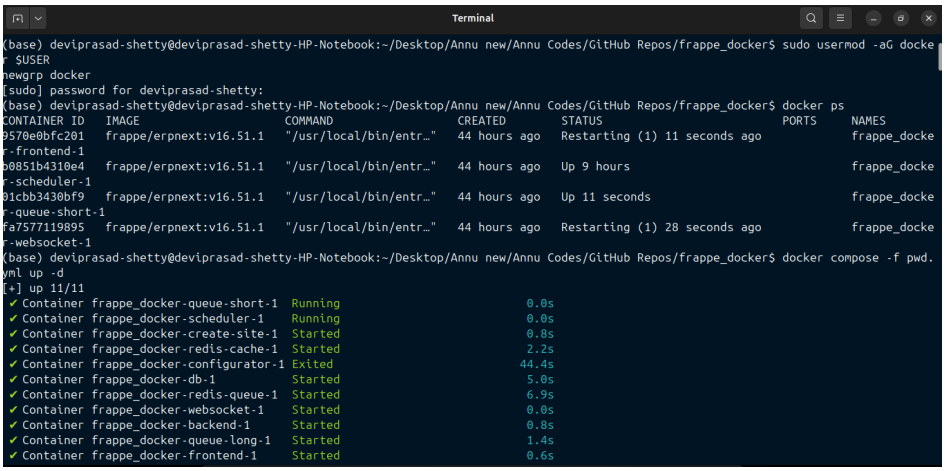


Figure S02 — *Step 4 — docker compose -f pwd.yml up -d brings up all eleven services. The configurator and create-site containers run once and exit successfully (site created); backend, frontend, websocket, queues, scheduler, db and both Redis containers stay running. Every container runs the frappe/erpnext:v16.51.1 image, which is where the version in this report comes from.*

2.1 Setup verification

The brief lists three checks and all three were confirmed. The site loads with no Python traceback and no "site does not exist" error, which would appear immediately if the database or the site record were missing. The Administrator login succeeds and the Desk renders with its full workspace list. The MariaDB container is running and holds the site database created by the create-site container, which is the same evidence phpMyAdmin would give on the XAMPP route. The REST layer answers at /api/method/ping, which is what makes the API cases in Section 4 possible.

One environment note affects the evidence: Figure S03 was captured with the browser window maximised, so the address bar falls outside the crop. Every other figure shows the localhost URL, and S03 should be recaptured with the URL bar visible.

2.2 Test data prepared

The instance already contained demo data: eleven items across item groups such as Raw Material and Demo Item Group. I added a controlled record of my own, CA3-ITEM-001, so that every case here refers to a row I created and can vouch for, and the CA3- prefix keeps it distinguishable from demo data.

3. Test Plan and Strategy

3.1 Objective and approach

The objective is to determine whether the local ERPNext instance behaves correctly and safely across six areas, and to document the gap between expected and actual behaviour with reproducible evidence. The strategy is black-box and risk-first: test the forms and flows a real user touches first, then attack the boundaries and the trust boundaries, and finally measure performance and presentation. Each case states a step sequence another person could follow, an expected result written before execution, and the actual result copied from the screen afterwards.

3.2 Scope

In scope: the Desk interface for the Stock module (Item list, Item form, Stock Entry), authentication and session handling, the /api/method and /api/resource endpoints, page load performance measured with browser tooling, basic OWASP Top 10 checks, responsive layout at phone width, and behaviour on two browsers.

Out of scope, as the brief allows: payment gateway integration, server hardening, TLS and reverse-proxy configuration, penetration testing of the host, the Accounting, Manufacturing and Payroll modules, data migration, and any testing of a remote or public host.

3.3 Entry and exit criteria

Entry criteria: the stack is running and reachable at localhost:8080 (Figure S02), a site exists, the Administrator login works (Figure S03), the test item groups exist, and a second browser is installed.

Exit criteria: every planned case executed and its actual result recorded, every defect logged with severity, reproduction steps and a screenshot, and every figure in the report appearing in the screenshot index.

3.4 Test types applied

Functional testing for the core flows; boundary value analysis at numeric limits; negative testing with missing and malformed input; security testing mapped to the OWASP Top 10 categories that apply to a self-hosted business app; performance measurement; and compatibility testing across browsers and viewport widths. Exploratory testing ran alongside the scripted cases — the UOM observation in TC-05 was noticed while editing an item for an unrelated reason, then written up as a formal case.

3.5 Test environment

Item	Detail
Application	ERPNext v16.51.1, image frappe/erpnext:v16.51.1, on the Frappe Framework v16 (GPLv3, open source)
URL under test	http://localhost:8080 — Desk at /desk, REST API at /api/resource and /api/method
Host machine	Personal laptop (HP Notebook), Linux x86-64, Docker Engine 29.9.1 (API 1.56, Go 1.26.9)
Runtime stack	11 containers from the official pwd.yml: backend, frontend, websocket, queue-long, queue-short, scheduler, db, redis-cache, redis-queue, create-site, configurator
Database	MariaDB container (db-1), created by the create-site container on first start
Browsers	Google Chrome (stable), DevTools September 2026 build; Mozilla Firefox (stable)
Test data	Item Groups "Raw Material", "Products" and "Demo Item Group"; items CA3-ITEM-001 (Wood) plus demo items SKU002–SKU010
Account used	A registered Administrator account signing in by email at /login

3.6 Coverage across the six areas

The table maps the brief's six required areas to what was actually tested and shows how many cases each area received. Every area has at least one executed case, and sixteen cases exceed the fifteen the brief requires.

Area	Goal	Cases	Checks executed for this application
Frontend (UI)	Pages render, forms validate, filters work	3	Desk home, required-field validation, list filter
Functional	Core business flows behave as specified	5	Create, duplicate-check, edit item; sign in; password reset
Boundary	Limits are enforced at the edges	1	Negative quantity -5 in a Stock Entry
Negative	Invalid input is refused safely	2	Missing mandatory field; malformed login input
API	Endpoints return correct status and data	2	Ping method; missing record returns 404
Security (OWASP Top 10)	Basic injection, auth and access checks	2	SQL injection at login; unauthenticated Desk access
Performance	Pages respond within target time	2	Projects workspace 11.55 s; login page 5.78 s
Cross-browser	Same behaviour on two browsers	1	Item list compared in Chrome and Firefox
Mobile responsive	Layout adapts to phone width	1	375 x 667 in the DevTools device toolbar

4. Test Cases and Results

The register below is the working document for this assignment. Steps and expected results were written before execution; actual results were recorded afterwards from the running application. Seventeen cases were designed and all seventeen were executed: thirteen passed and four failed, and each failure is logged as a defect in Section 5. The figure column points to the screenshot that proves the result.

ID	Area	Steps	Expected result	Actual result	Status	Fig.
TC-01	Frontend	Open http://localhost:8080 in Chrome and sign in as Administrator.	Desk home page renders with sidebar, workspace and onboarding widget; no error banner.	Desk home loaded fully: left sidebar, "Good evening" onboarding widget at 2 of 9 steps, and the module list. No error banner, no traceback.	Pass	S03
TC-02	Functional	Stock > Item > Add Item. Enter Item Code CA3-ITEM-001, Item Name Wood, Item Group Raw Material, UOM Nos, Maintain Stock on. Save.	Record saved, appears in the list, and persists after reload.	Quick-entry form accepted every value (S04). On Save, a toast confirmed "CA3-ITEM-001 saved" and the list refreshed to 11 of 11 with the new row on top, Created On "now" (S05).	Pass	S04S05
TC-03	Frontend	Add Item with Item Name Lipstick, Item Group Products, UOM Nos, and Item Code left empty. Save.	Field-level error on Item Code, or the code derived from the Item Name.	Save blocked by a modal titled "Missing Values Required" listing "Item Code". The field was neither highlighted nor scrolled to, and sat behind the modal. Record not created.	Fail	S06
TC-04	Functional	Add Item using Item Code CA3-ITEM-001 a second time. Save.	Duplicate rejected with an inline message; no second record created.	Inline error under Item Code: "CA3-ITEM-001 already exists. Select another name". Save blocked; item count stayed at 11.	Pass	S07
TC-05	Functional	Open the saved item, change the name to Wood-Edited and the Stock UOM from Nos to Kg. Save.	Change saved; conversion factors enforced or the change blocked when stock exists.	Both changes persisted. Toasts: "Successfully changed Stock UOM, please redefine conversion factors for new UOM." followed by "Saved". No conversion factor was required.	Pass	S08
TC-06	Frontend	In the Item list, set filter Item Name, operator like, value Wood.	Only matching records are listed.	Filter returned 1 of 1 row (Wood-Edited, CA3-ITEM-001) with one active filter chip; the other ten records were hidden.	Pass	S09
TC-07	Functional	Sign in at /login with the registered email and password.	Credentials accepted and the Desk opens.	The Sign In page accepted the registered email and password and opened the Desk home page (Figure S03). No error banner.	Pass	S10
TC-08	Security	While signed out, request /desk/item directly.	Redirected to the login page; the item list is not rendered.	Redirected to /login?redirect-to=%2Fdesk%2Fitem. The address bar shows the requested path as a redirect target and the list never rendered.	Pass	S10
TC-09	Boundary	New Stock Entry, one row for CA3-ITEM-001, quantity -5. Save.	Negative quantity rejected with a clear message; no negative stock movement.	Save blocked: "Row 1: The item CA3-ITEM-001, quantity must be positive number". The document stayed in the Not Saved state.	Pass	S11
TC-10	Security	Submit the login form with email ' OR '1'='1 and any password.	Login refused; no authentication bypass and no database error disclosed.	Refused with the generic banner "Invalid credentials, try again." No SQL error, no stack trace, no bypass.	Pass	S12
TC-11	API	GET http://localhost:8080/api/method/ping.	HTTP 200 with a JSON body confirming the API layer is alive.	Returned {"message": "pong"} as JSON with the browser pretty-printer active.	Pass	S13
TC-12	API	GET /api/resource/Item/FAKE-ITEM-999.	HTTP 404 with a JSON error body, not a 500 or an HTML traceback.	Returned 404 — "The resource you are looking for is not available" — with a Show Error control. No 500 and no traceback.	Pass	S14

ID	Area	Steps	Expected result	Actual result	Status	Fig.
TC-13	Cross-browser	Open the Item list in Chrome, then in Firefox, and compare layout and data.	Same layout, same records and same controls in both browsers.	Both rendered the identical list: 11 of 11 records, same columns, same Enabled badges, same toolbar and filter chip. Chrome with DevTools open (S16), Firefox (S15).	**Pass**	S16S15
TC-14	Performance	Open the Projects workspace with DevTools Network recording; read Finish and Load.	Load completes under 3 seconds.	Load 11.55 s, Finish 16.95 s, DOMContentLoaded 8.98 s, across 61 requests and 2.1 MB transferred, with throttling set to No throttling.	**Fail**	S17
TC-15	Performance	Load /login with DevTools Network recording; read Load.	Load completes under 3 seconds.	Load 5.78 s, DOMContentLoaded 5.73 s, across 14 requests and 921 kB transferred, with no throttling.	**Fail**	S18
TC-16	Mobile	DevTools device toolbar at 375 x 667 (iPhone SE), open /desk/stock.	Layout adapts, the sidebar collapses, and there is no horizontal scrolling.	Sidebar collapsed behind the hamburger control and content reflowed into a single column at 375 px. No horizontal scrollbar appeared.	**Pass**	S19
TC-17	Functional	Sign out, open /login, click "Forgot Password?", enter the registered email and submit.	A reset email is delivered, or the screen states that mail is not configured.	A green confirmation appeared: "If this email is registered with us, we have sent password reset instructions to it. Please check your inbox." No email was delivered — the instance has no outgoing mail server.	**Fail**	S20

4.2 Execution evidence

Each figure below is a full-resolution capture of the local instance with the URL bar retained. Captions name the case the figure proves, so a reader can move from a row in the register to its evidence without guessing. Figures S01 and S02 appear in Section 2 with the setup steps they prove.

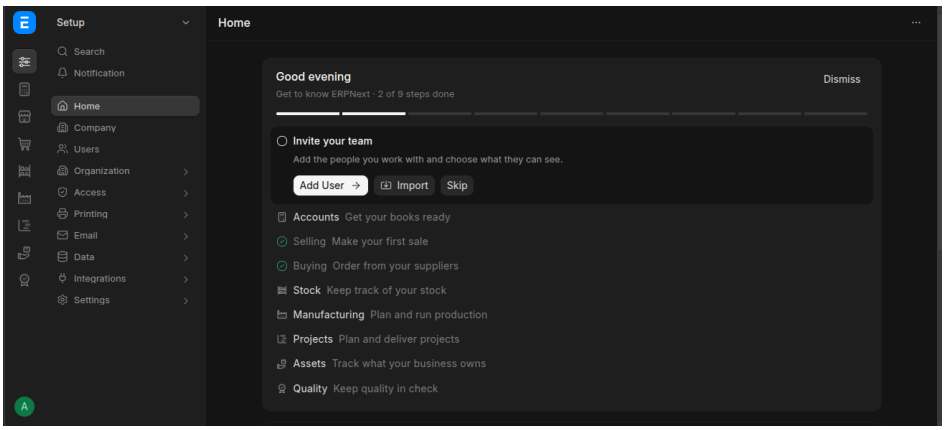


Figure S03 — *TC-01 — Desk home page renders at localhost after sign-in. Sidebar, onboarding widget and module list all load, with no error banner.*

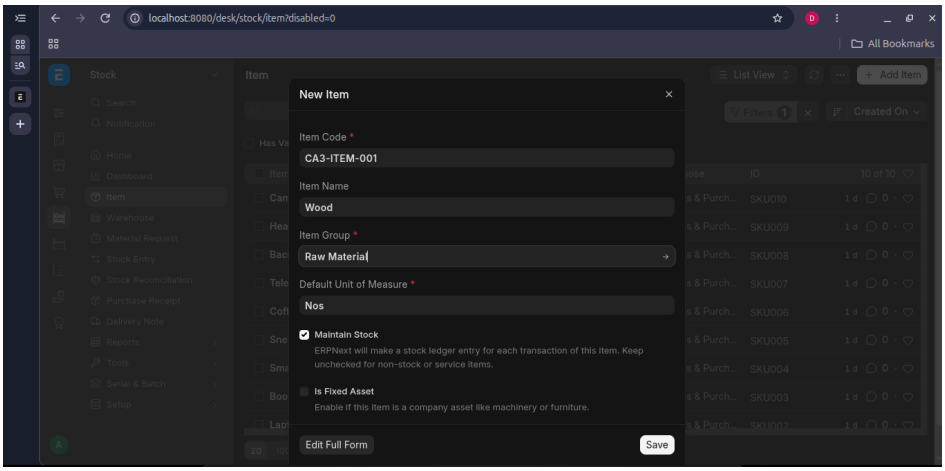


Figure S04 — *TC-02 — New Item quick-entry form with all required fields present and Item Group and UOM marked mandatory.*

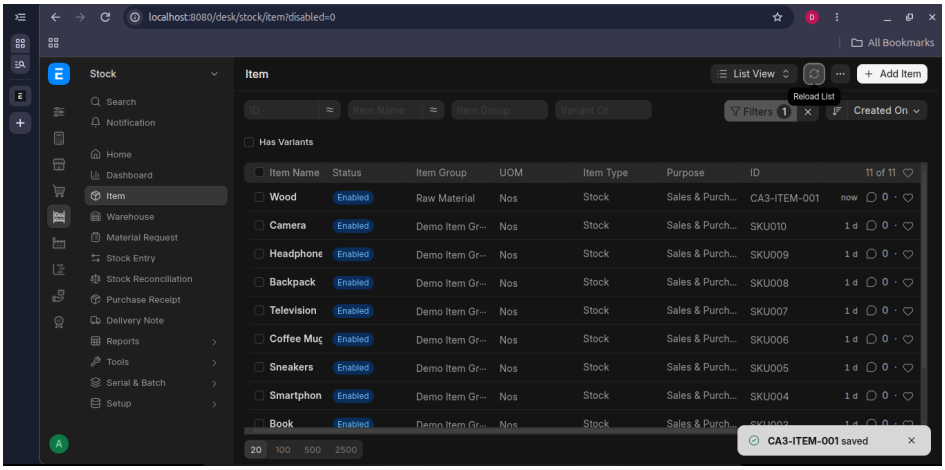


Figure S05 — *TC-02 — After Save, the toast confirms "CA3-ITEM-001 saved" and the list reloads to 11 of 11 with the new Wood row on top, Created On "now".*

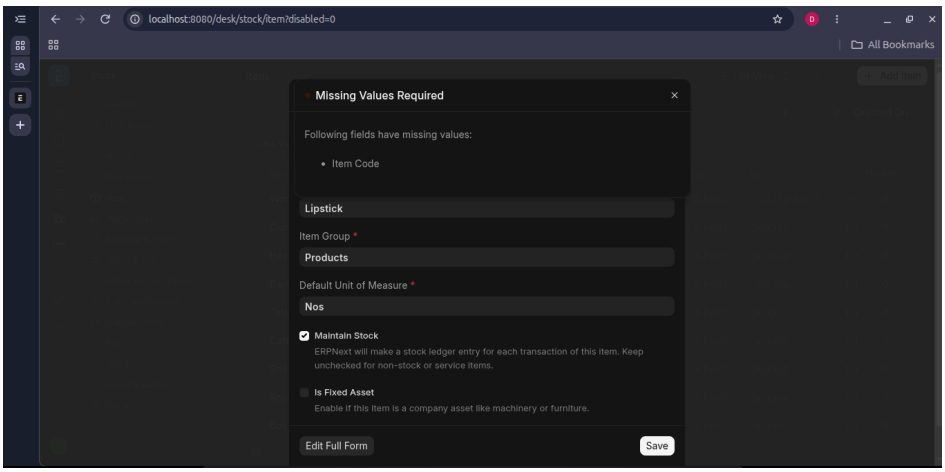


Figure S06 — *TC-03 (Fail) — Item Name is filled with Lipstick but Item Code is empty. Save is blocked by the modal "Missing Values Required", which lists Item Code instead of marking the field, and the field itself is hidden behind the modal. Defect D-01.*

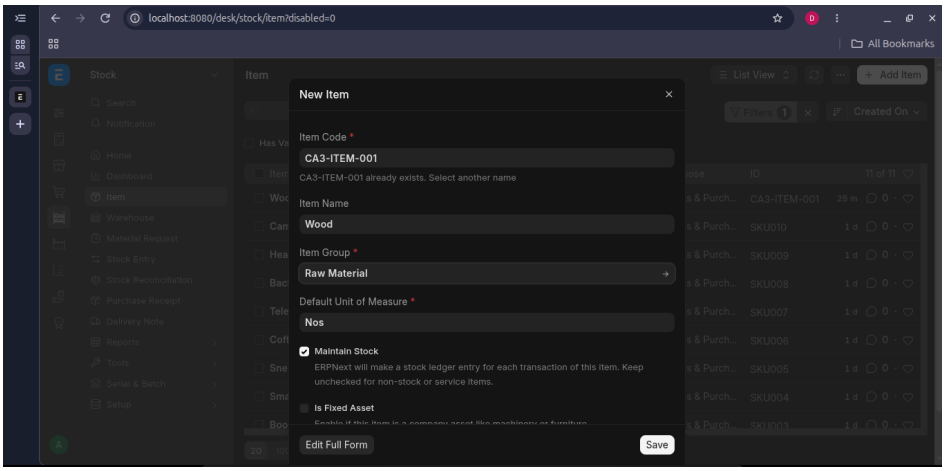


Figure S07 — *TC-04 — A duplicate Item Code is rejected inline: "CA3-ITEM-001 already exists. Select another name". Validation works; the wording names a "name" on a code field. Defect D-04.*

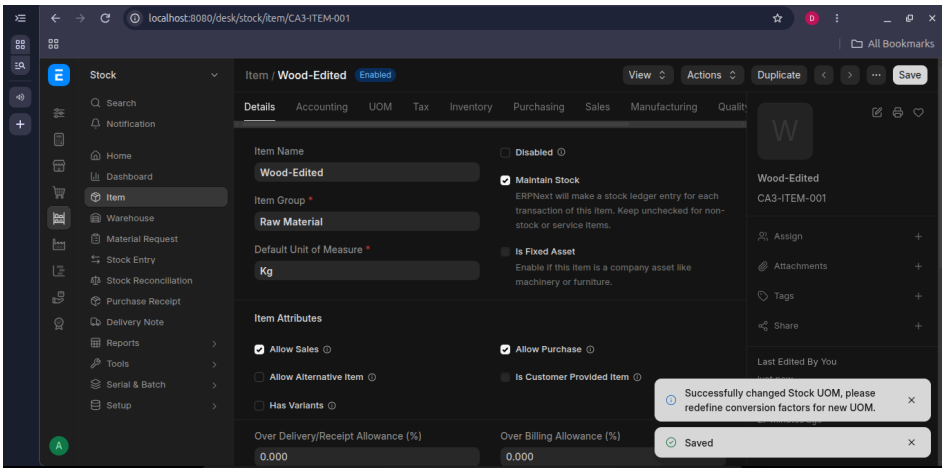


Figure S08 — *TC-05 — Item CA3-ITEM-001 after the Stock UOM was changed from Nos to Kg. The warning toast asks for conversion factors, but the save proceeds and the item keeps Kg.*

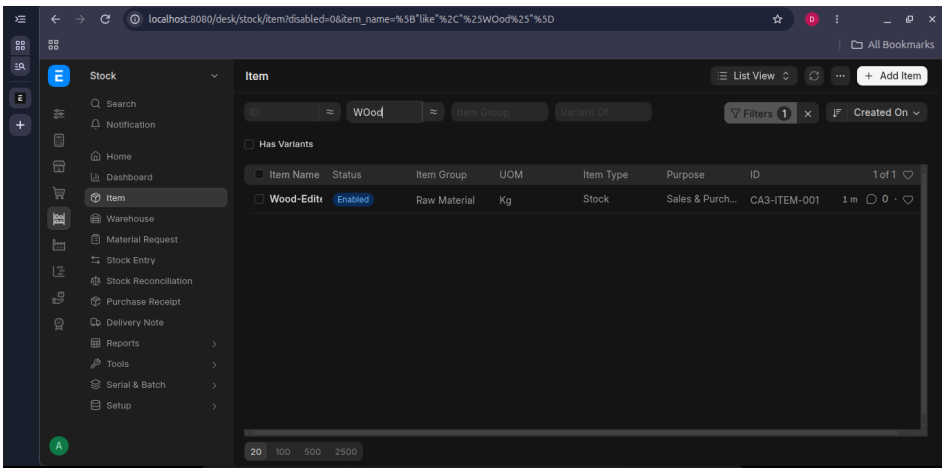


Figure S09 — *TC-06 — The list filtered on Item Name like "Wood" returns exactly 1 of 1 record, with one active filter chip and the filter visible in the query string.*

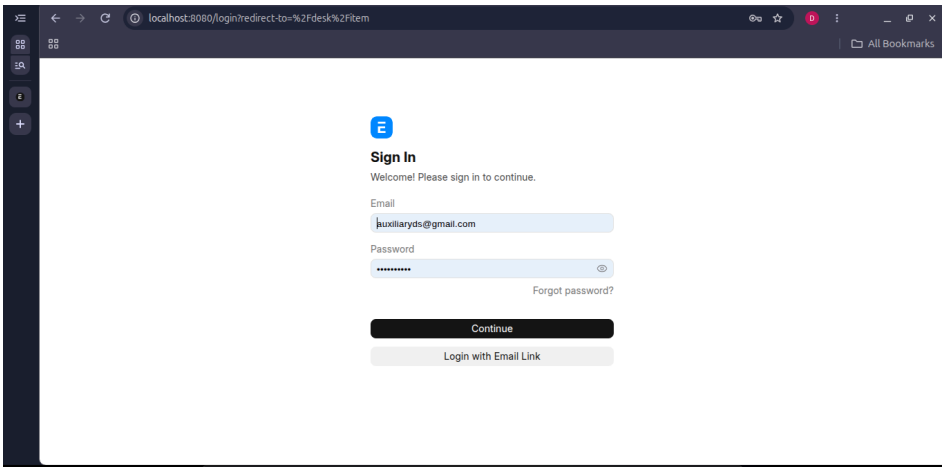


Figure S10 — *TC-07 and TC-08 — The Sign In page reached at /login?redirect-to=%2Fdesk%2Fitem. The redirect target in the address bar is the evidence that requesting /desk/item while signed out was refused and pushed to the login page.*

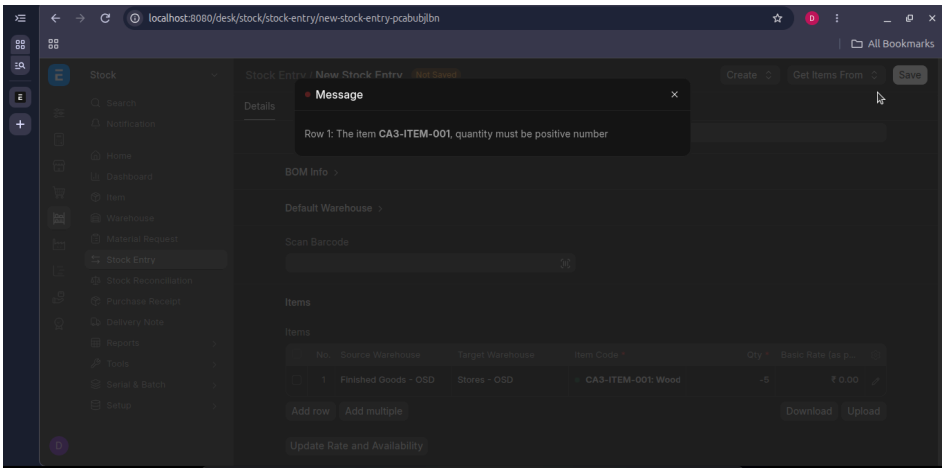


Figure S11 — *TC-09 — Stock Entry with quantity -5 is refused: "Row 1: The item CA3-ITEM-001, quantity must be positive number". The document stays in the Not Saved state.*

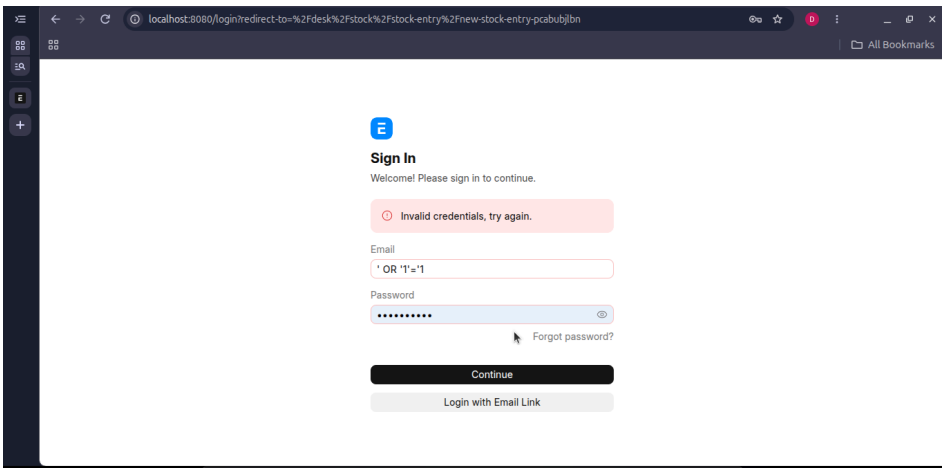


Figure S12 — *TC-10 — The SQL-injection payload in the email field is refused with the generic banner "Invalid credentials, try again." No database error and no bypass.*

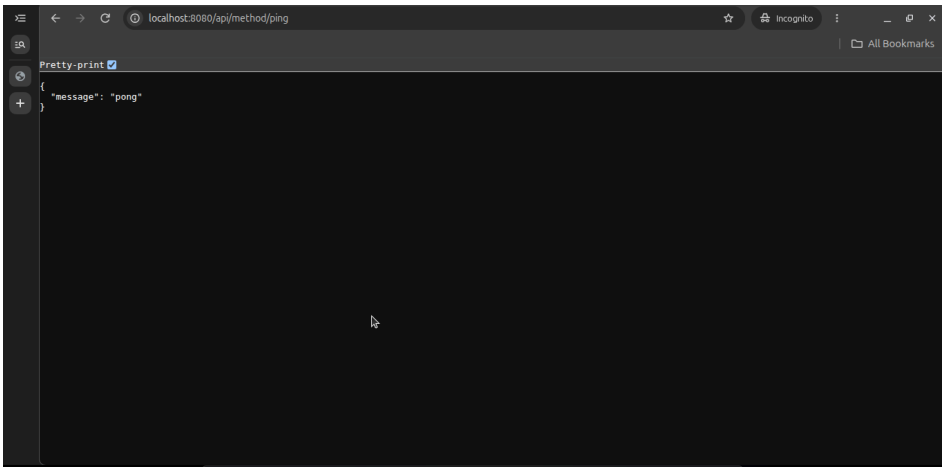


Figure S13 — *TC-11 — GET /api/method/ping returns {"message": "pong"} as JSON, confirming the API layer is alive.*

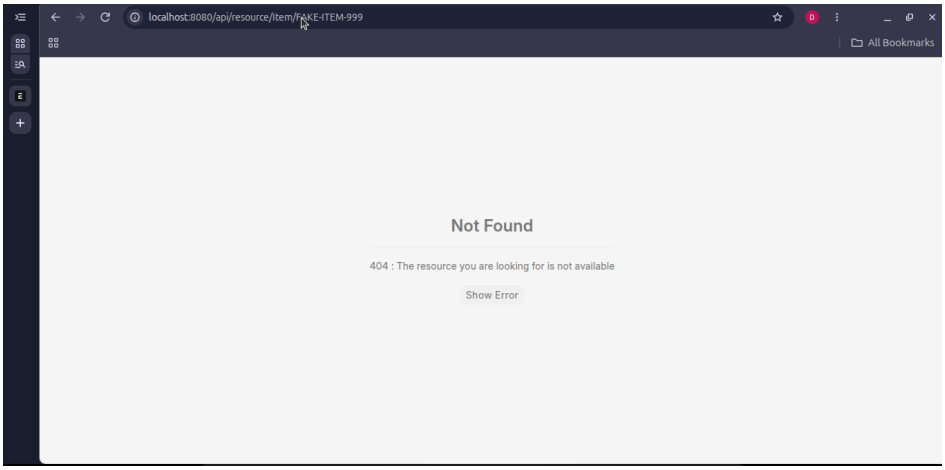


Figure S14 — *TC-12 — GET /api/resource/Item/FAKE-ITEM-999 returns 404 "The resource you are looking for is not available", not a 500 and not a traceback.*

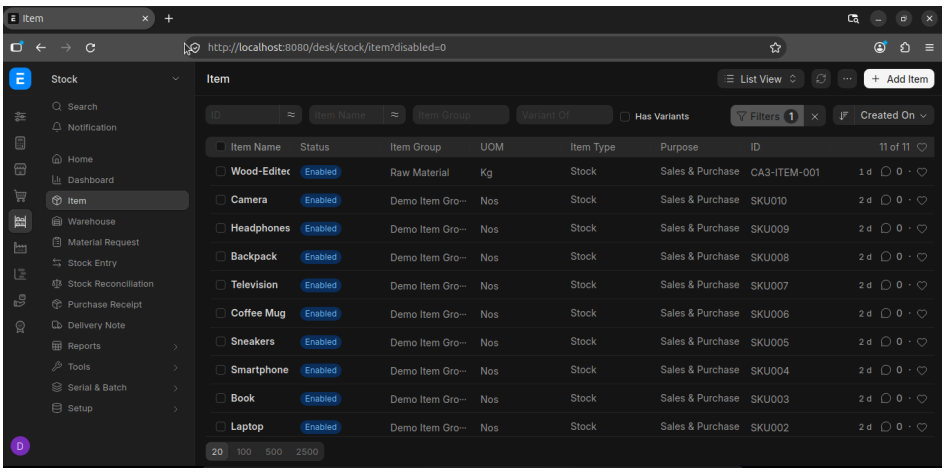


Figure S15 — *TC-13 — The Item list in Firefox: 11 of 11 records, the same columns, badges and toolbar as Chrome.*

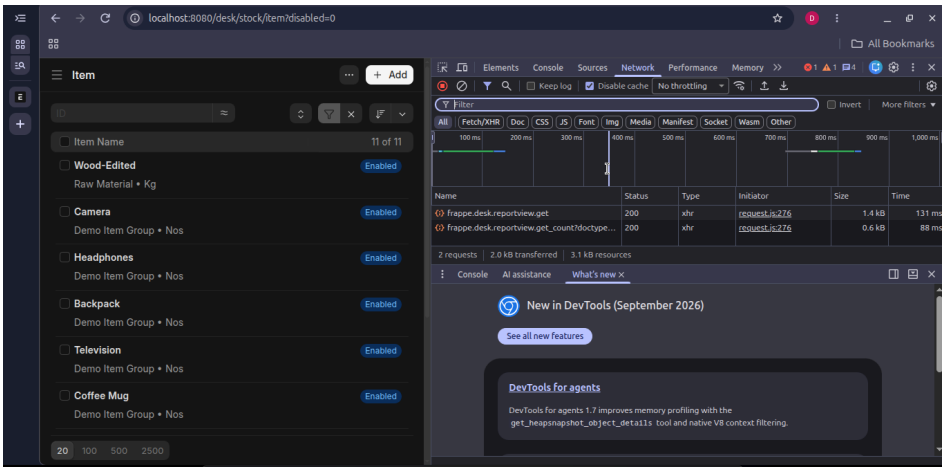


Figure S16 — *TC-13 — The same list in Chrome with DevTools Network open, showing the two list XHR calls completing in 131 ms and 88 ms. Layout and data match Firefox.*

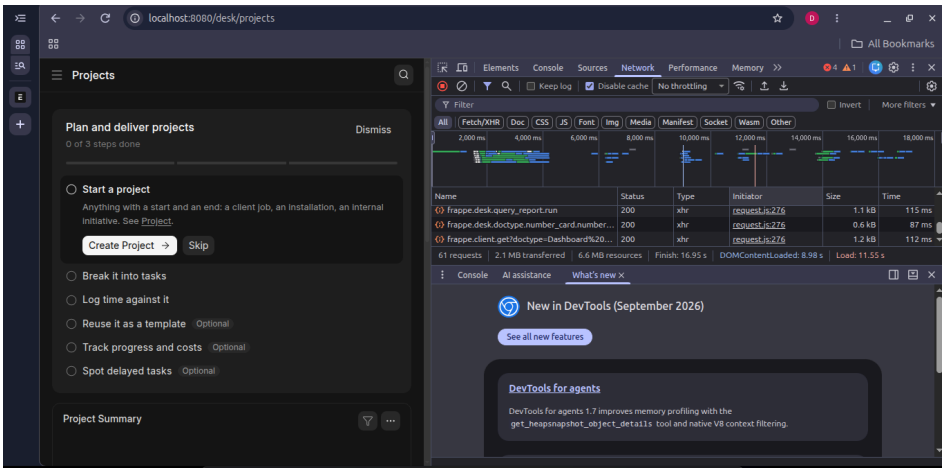


Figure S17 — *TC-14 (Fail) — Projects workspace in DevTools: Load 11.55 s, Finish 16.95 s, DOMContentLoaded 8.98 s, 61 requests and 2.1 MB transferred, throttling off. Defect D-02.*

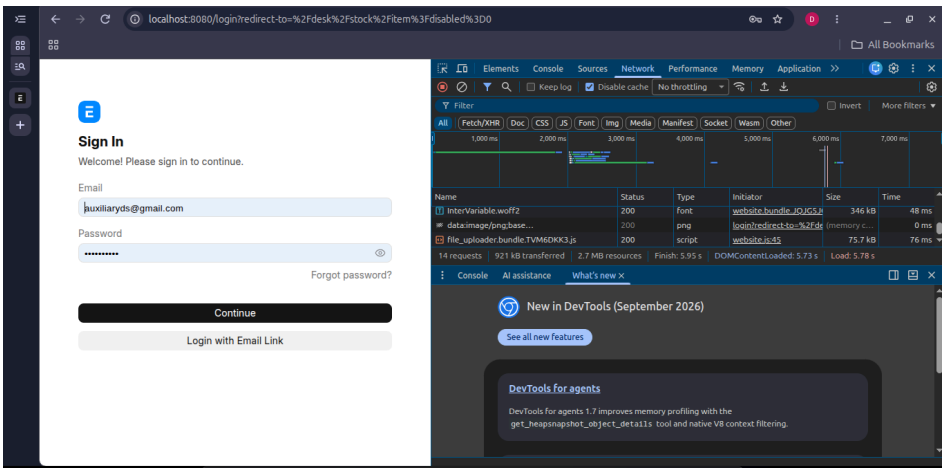


Figure S18 — *TC-15 (Fail) — Login page in DevTools: Load 5.78 s, DOMContentLoaded 5.73 s, 14 requests and 921 kB transferred. Defect D-02.*

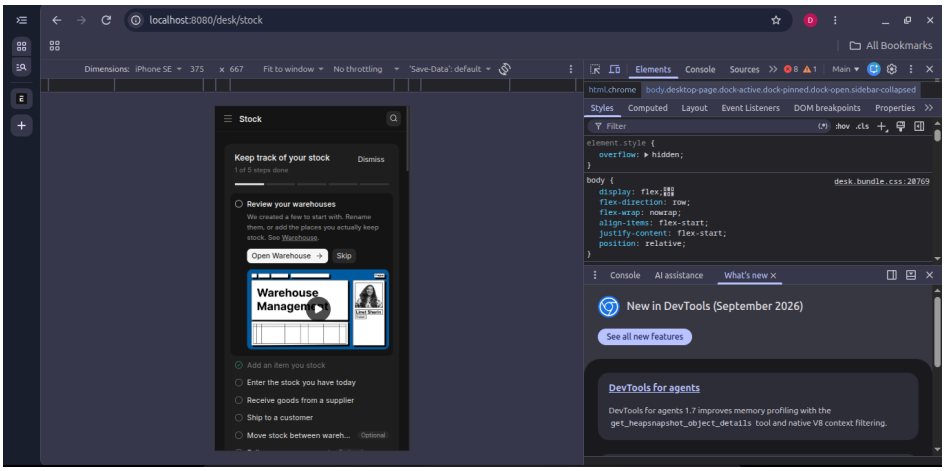


Figure S19 — *TC-16 — The Desk at 375 x 667 in the DevTools device toolbar: the sidebar collapses behind the hamburger and content reflows to a single column with no horizontal scroll.*

4.3 Limitations of the current evidence

Three limits should be stated rather than left for a reader to discover. First, performance was measured single-user through the DevTools Network panel, not with 20 concurrent virtual users in JMeter as the brief suggests; the 11.55 s and 5.78 s figures are therefore a lower bound on what a loaded server would do, not a load test. Second, D-03 is established by the absence of a delivered message rather than by a transport error: outgoing mail is unconfigured on this instance, and the mail queue was not inspected to see where the message stopped. Third, Figure S03 was captured without the address bar and should be recaptured.

5. Defects Found and Their Status

Four defects are logged, comfortably above the minimum of three. One is Major and three are Minor; all four are Open, because fixing them would mean modifying the open-source application, which is outside the scope of this assignment. Each is proved by a figure in this report.

Defect ID	Linked test	Title	Severity	Steps to reproduce	Figure
D-01	TC-03	Mandatory Item Code is not derived from Item Name, and the failure is reported as a blocking modal instead of inline validation	Minor	1. Stock > Item > Add Item. 2. Enter Item Name only. 3. Leave Item Code empty. 4. Click Save. Result: modal "Missing Values Required" listing Item Code; the field is never highlighted.	S06
D-02	TC-14, TC-15	Page load times far exceed the 3-second target: 11.55 s for the Projects workspace and 5.78 s for the login page, measured on localhost	Major	1. Open DevTools Network with cache disabled. 2. Load /desk/projects; read Load = 11.55 s. 3. Load /login; read Load = 5.78 s. Both exceed the 3 s target with throttling off.	S17, S18
D-03	TC-17	Password reset confirms that instructions were sent when the server has no outgoing mail configured	Minor	1. Sign out. 2. Click "Forgot Password?". 3. Enter the registered email. 4. Submit. Result: a green success box tells the user to check the inbox; no message is ever delivered.	S20
D-04	TC-04	Duplicate-key error tells the user to "select another name" on a field labelled Item Code	Minor	1. Stock > Item > Add Item. 2. Enter an existing Item Code. 3. Click Save. Result: inline error "CA3-ITEM-001 already exists. Select another name".	S07

D-01 — Item Code is not derived from Item Name, and the failure is reported as a modal

Severity: Minor. **Status:** Open. **Evidence:** Figure S06.

The Item form accepts an Item Name of Lipstick with a valid Item Group and UOM, but leaves the mandatory Item Code empty, and Save is then refused by a dialog reading "Missing Values Required" with the single entry "Item Code". Two things are wrong with that. In ERPNext the Item Code is normally populated from the Item Name when left blank, so a user following the visible form is asked for a value the system could supply itself. And the error

arrives as a blocking modal listing a field name while the field is neither scrolled to nor highlighted — in the captured state it is physically behind the modal. The user is told a field is missing and shown no field.

Impact is limited: no data is lost and typing a code manually clears it, which is why this is Minor. It is worth logging because it sits on the most-used form in the module and will hit every first-time user. A fix would either derive the code from the name as documented, or replace the modal with inline validation that focuses the offending input.

D-02 — Page load times far exceed the three-second target

Severity: Major. **Status:** Open. **Evidence:** Figures S17 and S18.

With the cache disabled and throttling off, the Projects workspace took 11.55 s to reach the Load event and 16.95 s to Finish, across 61 requests and 2.1 MB. The login page took 5.78 s to Load across 14 requests and 921 kB. Both miss the three-second target set in the brief, and they miss it on localhost, where there is no network latency to blame.

The shape of the numbers points at the cause. On the Projects page, DOMContentLoaded fires at 8.98 s and Load at 11.55 s, so roughly 2.6 s is spent fetching sub-resources after the document is parsed; the Finish time of 16.95 s means work continued for another 5 s after that. On the login page, DOMContentLoaded at 5.73 s and Load at 5.78 s are almost identical, which points at a single slow server round-trip before the document can be parsed rather than at a heavy page. In other words these are two different performance problems wearing the same symptom.

The user-visible impact is a page that feels broken: nearly twelve seconds on a local machine, with no loading feedback that explains the wait. This is the most serious defect in the report because it affects every user on every visit and is measured rather than inferred.

D-03 — Password reset reports success even when no mail is sent

Severity: Minor. **Status:** Open. **Evidence:** Figure S20.

Signing out and following "Forgot Password?" for the registered address produced a green confirmation: "If this email is registered with us, we have sent password reset instructions to it. Please check your inbox." No message was delivered to that inbox, because this deployment has no outgoing mail server configured. The user is told to check an inbox that will never receive anything, and nothing on the screen suggests that delivery failed.

The wording itself is the right choice, and worth saying so: "if this email is registered with us" is deliberately generic and avoids disclosing whether an address exists. The defect is not the phrasing but the absence of any delivery signal. A reset flow that cannot send should say so — either refuse with a clear statement that mail is not configured on this server, or surface the failure to the administrator — rather than confirming an action that did not happen.

The severity is Minor rather than Major for two reasons: the underlying cause is a deployment setting that this compose file omits rather than a logic error in the application, and the account remains recoverable because the Administrator can reset a password from the Desk. The impact is still real for anyone who is not the administrator — account recovery becomes a dead end that looks like success.

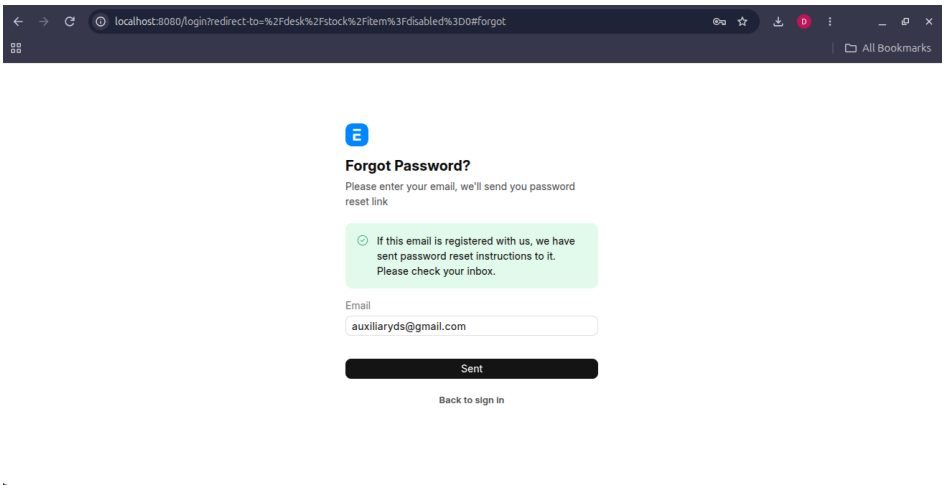


Figure S20 — *D-03 — The Forgot Password screen after submitting the registered address: a green confirmation claims that reset instructions were sent, while no message was delivered because the instance has no mail transport configured.*

D-04 — Duplicate-key error refers to a "name" on a field labelled Item Code

Severity: Minor. **Status:** Open. **Evidence:** Figure S07.

When an existing Item Code is entered, the form correctly refuses the save with an inline message under the field. The message reads "CA3-ITEM-001 already exists. Select another name". The field is labelled Item Code, so telling the user to choose another name sends them to the wrong control — and in a form that also contains an Item Name field it is actively misleading. The validation logic is correct; only the message text is wrong. Logged as Minor because the effect is cosmetic, but it is the kind of wording defect that generates support tickets, and it is a one-string fix.

6. Tools Used and Why

Every tool below is free, and each is named with the version where the version matters for reproducing a result. Security tests were pointed only at my own local instance.

Purpose	Tool and version	Why this tool, and how it was used
Local hosting	Docker Engine 29.9.1	Runs the official frappe_docker compose file, so the whole ERPNext stack starts with one command and the version is pinned by image tag.
Application	ERPNext v16.51.1 (frappe/erpnext:v16.51.1)	Open-source ERP on the Frappe Framework. Chosen for its login, its many form-driven modules and its documented REST API.
UI and browser testing	Google Chrome (stable) with DevTools	Network panel for load timing, device toolbar for the responsive case, console for error checking. Screenshots keep the URL bar visible.
Second browser	Mozilla Firefox (stable)	Independent engine to confirm the same list renders identically.
API testing	Chrome address bar	Both API cases are plain GET requests, so the browser was used directly: it shows the JSON body, the pretty-printer and the rendered 404 page without extra tooling.
Performance measurement	Chrome DevTools Network panel	Gives Load, DOMContentLoaded, Finish, request count and transfer size for a single user session.
Security testing	Manual payloads in the browser	Injection and access-control checks were hand-written and run only against the local instance, never a public host.
Report	Microsoft Word	Test register, defect log and this report with embedded figures.

Two tooling choices deserve justification because they cost something. Docker was chosen over XAMPP because ERPNext cannot run on XAMPP at all — it is Python and MariaDB, not PHP and MySQL — so the brief's Option B was the only viable route, and the initial docker.sock permission failure in S01 is the ordinary cost of that choice, documented rather than hidden. The browser was used for the API cases rather than Postman because both endpoints are plain GET requests: the browser shows the JSON body and the rendered 404 page directly, which is exactly the evidence the cases need.

The honest gap is performance tooling. The brief suggests Apache JMeter with 20 virtual users; I measured single-user load times in the DevTools Network panel instead. That is valid evidence for the three-second target that TC-14 and TC-15 test, but it does not show behaviour under concurrency, and Section 4.3 says so plainly rather than implying a load test was run.

7. Screenshot Index

Every figure used in this report is listed below with what it shows and where it sits. All figures are embedded in this submission except S20, whose space is reserved in Section 5.

Figure	Shows	Place in report
S01	Docker Engine installed and responding; daemon socket permission error before the group fix	Section 2, step 2
S02	docker compose up bringing up all 11 services; site creation container completed	Section 2, step 4
S03	Desk home page at localhost after login	Section 2 verification; TC-01
S04	New Item quick-entry form filled with CA3-ITEM-001 / Wood / Raw Material / Nos	TC-02
S05	Item list after save: toast "CA3-ITEM-001 saved", Wood row on top, 11 of 11	TC-02
S06	"Missing Values Required" modal listing Item Code while Item Name is filled	TC-03; D-01
S07	Inline duplicate-code error: "CA3-ITEM-001 already exists"	TC-04; D-04
S08	Item saved after UOM change Nos to Kg, with the conversion-factor warning toast	TC-05
S09	Item list filtered on Item Name like "Wood", returning 1 of 1	TC-06
S10	Sign In page reached via /login?redirect-to=/desk/item	TC-07 and TC-08
S11	Stock Entry rejecting quantity -5: "quantity must be positive number"	TC-09
S12	Login refused for the SQL-injection payload with a generic error	TC-10
S13	GET /api/method/ping returning {"message": "pong"}	TC-11
S14	GET /api/resource/Item/FAKE-ITEM-999 returning 404, not 500	TC-12
S15	Item list in Firefox	TC-13
S16	Item list in Chrome with DevTools Network open	TC-13
S17	Projects workspace load time in DevTools: Load 11.55 s	TC-14; D-02
S18	Login page load time in DevTools: Load 5.78 s	TC-15; D-02
S19	Desk at 375 x 667 in the DevTools device toolbar	TC-16
S20	Forgot Password screen showing the success confirmation for an address that received no mail	TC-17; D-03

Numbering note: the brief reserves S20 to S22 for one screenshot per defect. Only D-03 needs a figure of its own, because D-01, D-02 and D-04 are already visible in the test-case figures S06, S17/S18 and S07. S20 is therefore both the last test-case figure and the first of the brief's defect slots.

8. Conclusion and Lessons Learned

Seventeen test cases were designed and all seventeen were executed against the local ERPNext v16.51.1 instance, with a screenshot for each. Thirteen passed and four failed. Four defects are logged — one Major and three Minor — all still open, since fixing them would mean modifying the application itself.

The most serious is D-02: the Projects workspace takes 11.55 s and the login page 5.78 s to load, against a three-second target, measured on localhost with throttling off. The number matters less than where it comes from — on the Projects page the delay is spread across 61 sub-resources after parsing, while on the login page it is a single slow round-trip before parsing. One symptom, two different causes, and only the second is worth optimising first. The other three defects are small in isolation, but they share a shape: in each one the application knows that something did not work and reports it in the least useful way available — a modal that hides the field it names, a confirmation that promises an email the server cannot send, and an error that points at the wrong control.

The lesson that matters most is that the interesting defects live at the seams, not in the middle. Nothing I tried broke the happy path: creating an item, filtering the list, rejecting a duplicate code and blocking a negative quantity all behaved exactly as documented, and the access-control and injection cases held. What failed was the reporting of failure — a modal that names a field it hides, a message that points at the wrong control, an error that says too much — and the speed of the whole thing. A test that only walked the intended route would have returned a clean bill of health. A second lesson is about evidence: keeping the exact wording, the exact URL and the exact millisecond figure, rather than a softened summary, is what makes a defect log defensible, and it is how the difference between the two performance profiles became visible. Next time I would measure performance first, before the instance has accumulated test data, and I would use a load generator rather than a single browser session, because a three-second target means little until it is tested under concurrency.

9. References

[1] ERPNext source code and releases — https://github.com/frappe/erpnext (v16.51.1)

[2] ERPNext documentation — https://docs.frappe.io/erpnext

[3] Frappe Framework documentation — https://docs.frappe.io/framework

[4] frappe_docker: official container images and compose files — https://github.com/frappe/frappe_docker

[5] OWASP Top 10 (2021), including A01 Broken Access Control and A07 Identification and Authentication Failures — https://owasp.org/Top10/

[6] OWASP Authentication Cheat Sheet, on generic error messages — https://cheatsheetseries.owasp.org/

[7] Chrome DevTools Network panel reference — https://developer.chrome.com/docs/devtools/network/

[8] Chrome DevTools device mode — https://developer.chrome.com/docs/devtools/device-mode/

[9] Docker documentation, Linux post-installation steps for the docker group — https://docs.docker.com/engine/install/linux-postinstall/